Open in Colab

Prelab 10.2 Machine Learning 3 - Real-time Sound Recognition for Classification

CNN model training and evaluation

```
# Let's check the installed tensorflow version
import tensorflow as tf

print('TensorFlow Version is', tf.__version__)
```

```
TensorFlow Version is 2.19.0
```

```
# Install required Python packages for this colab
!pip install librosa
!pip install matplotlib
!pip install seaborn
```

```
Requirement already satisfied: librosa in /usr/local/lib/python3.12/dist-packages (0.11.0)
Requirement already satisfied: audioread>=2.1.9 in /usr/local/lib/python3.12/dist-packages (from libro
Requirement already satisfied: numba>=0.51.0 in /usr/local/lib/python3.12/dist-packages (from librosa)
Requirement already satisfied: numpy>=1.22.3 in /usr/local/lib/python3.12/dist-packages (from librosa)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from librosa)
Requirement already satisfied: scikit-learn>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from li
Requirement already satisfied: joblib>=1.0 in /usr/local/lib/python3.12/dist-packages (from librosa) (
Requirement already satisfied: decorator>=4.3.0 in /usr/local/lib/python3.12/dist-packages (from libro
Requirement already satisfied: soundfile>=0.12.1 in /usr/local/lib/python3.12/dist-packages (from libr
Requirement already satisfied: pooch>=1.1 in /usr/local/lib/python3.12/dist-packages (from librosa) (1
Requirement already satisfied: soxr>=0.3.2 in /usr/local/lib/python3.12/dist-packages (from librosa) (
Requirement already satisfied: typing_extensions>=4.1.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: lazy_loader>=0.1 in /usr/local/lib/python3.12/dist-packages (from libro
Requirement already satisfied: msgpack>=1.0 in /usr/local/lib/python3.12/dist-packages (from librosa)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from lazy_loader>
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from po
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from pooch
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from s
Requirement already satisfied: cffi>=1.0 in /usr/local/lib/python3.12/dist-packages (from soundfile>=0
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.0->s
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from req
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from req
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matpl
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotli
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matp
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matp
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplo
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matpl
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from m
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateut
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from s
Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matpl
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotli
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matp
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matp
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplo
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matpl
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from m
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateut
```

2.1 Data loading and signal processing

Before we load data and analyze them, let's figure out the experiments for the training data collection. As discussed in the previous session (Prelab 10.1), we defined the machine's three states: (1) OFF, (2), ON in vacuuming, and (3) ON in air-leaking. The data collection was basically performed in the lab space in a quiet environment. Data was collected from the MTConnect sound stream in WAV format. The audio specifications, such as sampling rate, resolution, and so on, were the same as the MTConnect stream. The configuration of data collection is illustrated in Figure 7. To make a noisy environment, a loudspeaker was placed near the pump.

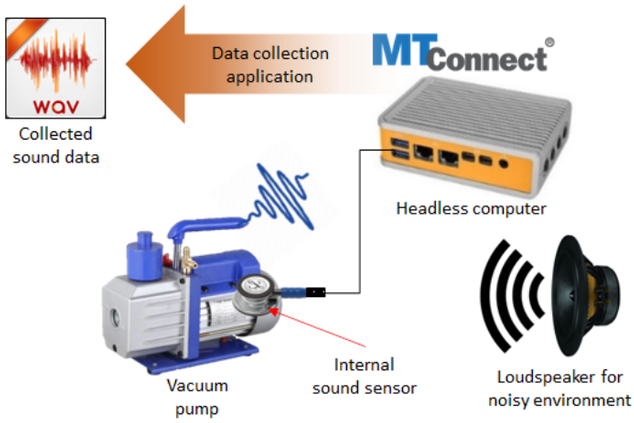


Figure 7 System configuration for sound data collection

Sound signals from the internal sound sensor for each state of the machine were collected for 1 minute. On top of that, simulating other environmental conditions, the same amount of data was collected in a loud environment in which a loudspeaker generated white noise. Therefore, we have three states data and total 6 sound data in Github repo. You will figure out the condition in the filename. Note that each data is 1 minute-long. Description for the filename and states is below.

- **OFF.wav**
- Machine is turned off, state 1 (OFF)
- **OFF_with_noise.wav**
- Machine is turned off, state 1 (OFF)
- **ON_vacuum.wav**
- Machine is on in vacuuming, state 2 (ON/Vacuuming)
- **ON_vacuum_with_noise.wav**
- Machine is on in vacuuming and noisy environment, state 2 (ON/Vacuuming)
- **ON_air_leaking.wav**
- Machine is on in air-leaking, state 3 (ON/Air-leaking)
- **ON_air_leaking_with_noise.wav**
- Machine is on in air-leaking and noisy environment, state 3 (ON/Air-leaking)

Let's listen to the sound of each case and then load data to analyze it.

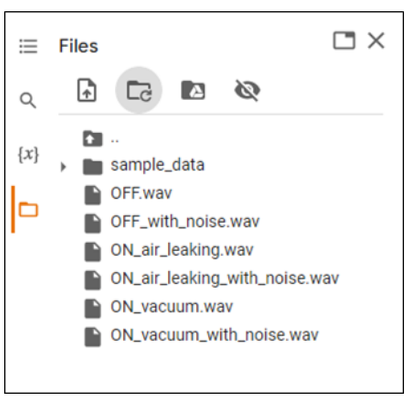
Loading data

We will use bash command in the code block below to load data in Colab session. Note that you can use Colab interface to upload data or mount your Google Drive instead.

```
# loading sound data in this Colab session by bash
# It takes a few seconds.
!wget -O OFF.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_data/OFF
!wget -O OFF_with_noise.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prela
!wget -O ON_vacuum.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_dat
!wget -O ON_vacuum_with_noise.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab16
!wget -O ON_air_leaking.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prela
!wget -O ON_air_leaking_with_noise.wav https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/
```

```
--2026-04-07 17:25:24--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/p
--2026-04-07 17:25:25--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/m
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133,
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connecte
HTTP request sent, awaiting response... 200 OK
Length: 5759020 (5.5M) [audio/wav]
Saving to: ‘ON_vacuum_with_noise.wav’

ON_vacuum_with_nois 100%[===================>]   5.49M  --.-KB/s    in 0.08s

2026-04-07 17:25:25 (71.6 MB/s) - ‘ON_vacuum_with_noise.wav’ saved [5759020/5759020]

--2026-04-07 17:25:25--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_
Resolving github.com (github.com)... 140.82.113.4
Connecting to github.com (github.com)|140.82.113.4|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/
--2026-04-07 17:25:25--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/p
--2026-04-07 17:25:26--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/m
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133,
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connecte
HTTP request sent, awaiting response... 200 OK
Length: 5760044 (5.5M) [audio/wav]
Saving to: ‘ON_air_leaking.wav’

ON_air_leaking.wav  100%[===================>]   5.49M  --.-KB/s    in 0.09s

2026-04-07 17:25:26 (60.1 MB/s) - ‘ON_air_leaking.wav’ saved [5760044/5760044]

--2026-04-07 17:25:26--  https://github.com/purduelamm/purdue_me597_iiot/blob/main/lab/lab10/prelab_
Resolving github.com (github.com)... 140.82.113.3
Connecting to github.com (github.com)|140.82.113.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab10/prelab_data/
--2026-04-07 17:25:27--  https://github.com/purduelamm/purdue_me597_iiot/raw/refs/heads/main/lab/lab
Reusing existing connection to github.com:443.
HTTP request sent, awaiting response... 302 Found
Location: https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/main/lab/lab10/p
--2026-04-07 17:25:27--  https://raw.githubusercontent.com/purduelamm/purdue_me597_iiot/refs/heads/m
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.109.133,
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443... connecte
HTTP request sent, awaiting response... 200 OK
Length: 5759020 (5.5M) [audio/wav]
Saving to: ‘ON_air_leaking_with_noise.wav’
```

After running the cell above, Files of your Colab session look like the capture below.



Let's play each data in the following code blocks and perform Task 2.1.

```
from IPython.display import Audio, display

# Filenames for each data
OFF = 'OFF.wav' # OFF state filename
OFF_with_noise = 'OFF_with_noise.wav' # OFF in noisy environment
ON_vacuum = 'ON_vacuum.wav' # ON in vacuuming
ON_vacuum_with_noise = 'ON_vacuum_with_noise.wav' # ON in vacuuming and noisy environment
ON_air_leaking = 'ON_air_leaking.wav' # ON in air-leaking
ON_air_leaking_with_noise = 'ON_air_leaking_with_noise.wav' # On in air-leaking and noisy environment
```

```
print(OFF)
display(Audio(OFF))
```

```
OFF.wav
```

0:00 / 1:00

```
print(OFF_with_noise)
display(Audio(OFF_with_noise))
```

```
OFF_with_noise.wav
```

0:00 / 0:59

```
print(ON_vacuum)
display(Audio(ON_vacuum))
```

```
ON_vacuum.wav
```

0:00 / 1:00

```
print(ON_vacuum_with_noise)
display(Audio(ON_vacuum_with_noise))
```

```
ON_vacuum_with_noise.wav
```

0:00 / 0:59

```
print(ON_air_leaking)
display(Audio(ON_air_leaking))
```

```
ON_air_leaking.wav
```

0:00 / 1:00

```
print(ON_air_leaking_with_noise)
display(Audio(ON_air_leaking_with_noise))
```

```
ON_air_leaking_with_noise.wav
```

0:00 / 0:59

Task 2.1

After listening to the sound according to each state (condition), do you think you can figure out the state based only on sound? Describe how you hear the sounds depending on the states.

Yes, it is possible to distinguish the compressor's operational states using only sound because each condition produces distinct acoustic patterns in terms of frequency, amplitude, and consistency.

Feature	OFF	OFF (Noise)	ON (Vacuum)	ON (Vacuum + Noise)	ON (Air Leak)	ON (Air Leak + Noise)
Sound Level	Silent	Low background	Strong	Strong + disturbance	Medium-high	Medium + disturbance
Sound Type	No sound	Ambient noise	Smooth humming	Humming + noise	Hissing / leaking	Hissing + noise
Frequency	None	Random low	Low-mid	Mixed	Higher freq	High + mixed
Amplitude	Zero	Low	Stable high	Slight fluctuation	Fluctuating	More fluctuation
Pattern	None	Random	Consistent	Slightly irregular	Irregular	Highly irregular

Signal processing

You definitely tell the differences between OFF and ON states. In addition, you probably "feel" different sounds between ON/vacuuming and ON/air-leaking states. Using signal processing, let's analyze data to see how signals are different. To analyze audio, we will mainly use Python librosa package. The librosa is a Python package for music and audio analysis.

Figure 8 illustrates the procedure of development of a machine learning model in a typical and simple. As we did in Lab 8 and Lab 9, there are many ways to perform signal processing. Even we can use raw data for a feature for the training model input. This process, making input data for a machine learning model, is normally called feature extraction or feature engineering. When determining the feature, we have to consider many aspects, such as data flow, dimension, coherence, efficiency, effectiveness, representativeness, performance, and so on. As a practice in this lab, we will only use one feature, however, it is highly recommended to try other features for training the model. In addition, upon the evaluation, we are able to optimize not only feature extraction method but also machine learning model (hyper parameters, architecture, and so on) for the best performance.

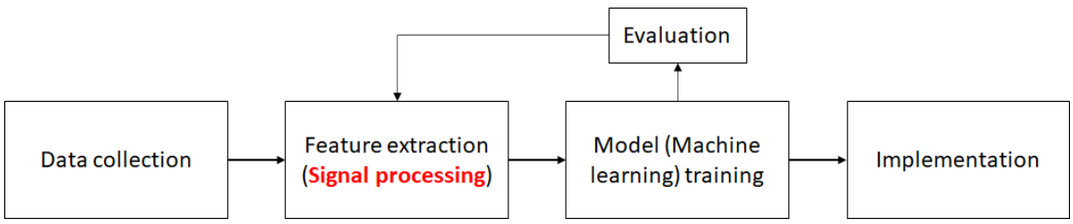


Figure 8 Development procedure of a typical and simple machine learning model

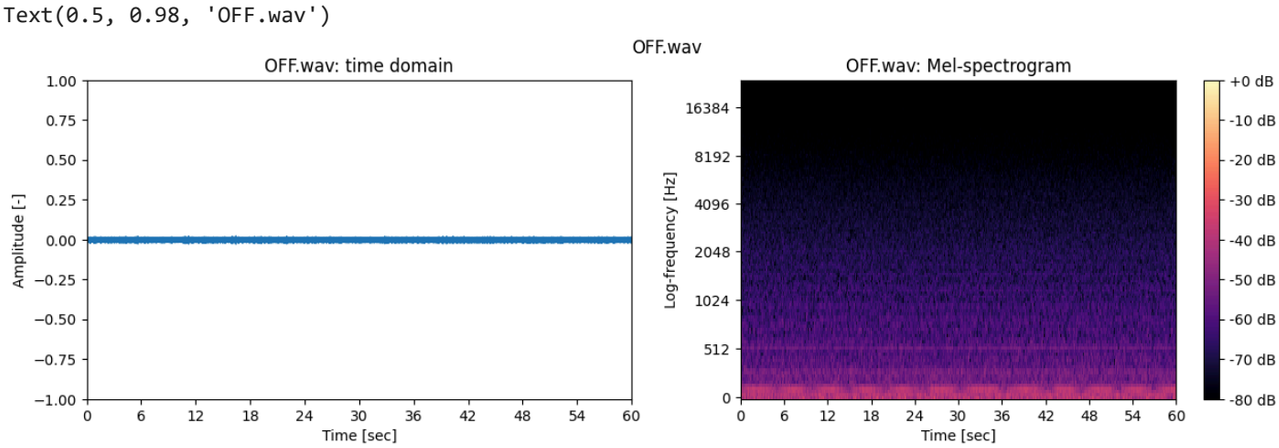
As a feature for the model, we will use Mel-spectrogram. The Mel-scale is a human perceived frequency of a pure tone to its actual measured frequency. Humans are much better to tell subtle changes at low frequencies than at high frequencies. Mel-spectrogram is more suitable for human auditory sensing that indicates the linear distirubtion under 1,000 Hz and the logarithm growth above 1,000 Hz, which result in many ASR (automatic speech recognition) techniques adopt the Mel-scale as a step of feature extractions.

In the following code blocks, let's visualize data and practice Mel-spectrogram process. Of course, you can create your own method for the signal processing using Python basic functions, NumPy, SciPy, and so on. However, we will take advantage of the librosa package. Visit librosa.feature.melspectrogram and Understanding the Mel Spectrogram to figure out the method and Mel-spectrogram.

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
import numpy as np

N_FFT = 2048 # Number of FFT points, because MTConnect stream chunk size is 2048. This is a constant
y_off, sr = librosa.load(OFF, sr=48000) # y_off is audio data in off state and sr is sampling rate
M_off = librosa.feature.melspectrogram(y=y_off, sr=sr, n_fft=N_FFT, hop_length=int(N_FFT/4), win_leng
```

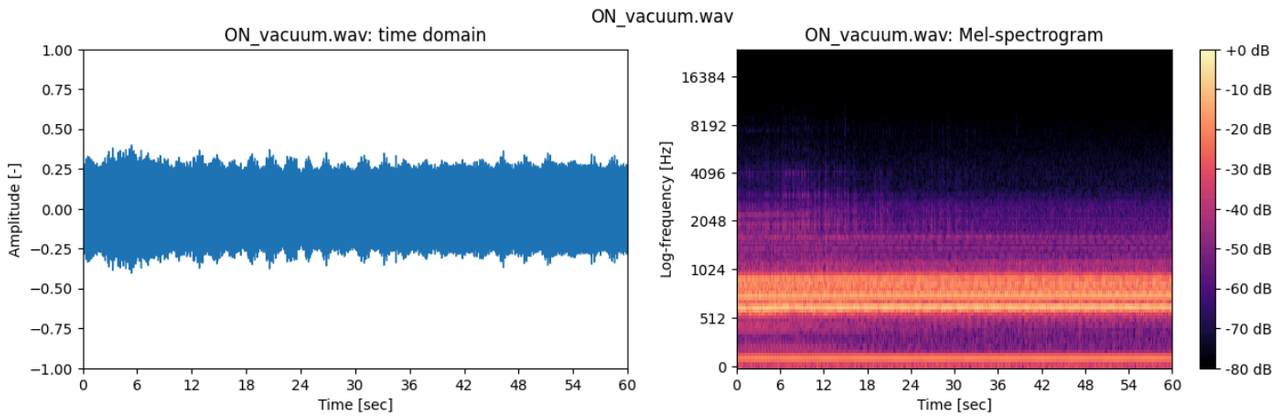
```
# This example plots time-domain (raw signal) on the left-hand side and spectrogram on the right-han
# Note that legend (color) means relative signal intensity in dB unit in the spectrogram.
# Also, in the spectrogram, the y-axis (frequency) is log scale.
fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))
librosa.display.waveshow(y_off, sr=sr, ax=ax[0]) # time domain (raw data) plot
img = librosa.display.specshow(librosa.power_to_db(2*abs(M_off)/N_FFT, ref=1), ax=ax[1], x_axis='tim
ax[0].set(title=OFF+": time domain", ylim=[-1, 1], xlim=[0, 60], ylabel="Amplitude [-]", xlabel="Tim
ax[1].set(title=OFF+": Mel-spectrogram", ylabel="Log-frequency [Hz]", xlabel="Time [sec]")
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(OFF)
```

```
Text(0.5, 0.98, 'OFF.wav')
```



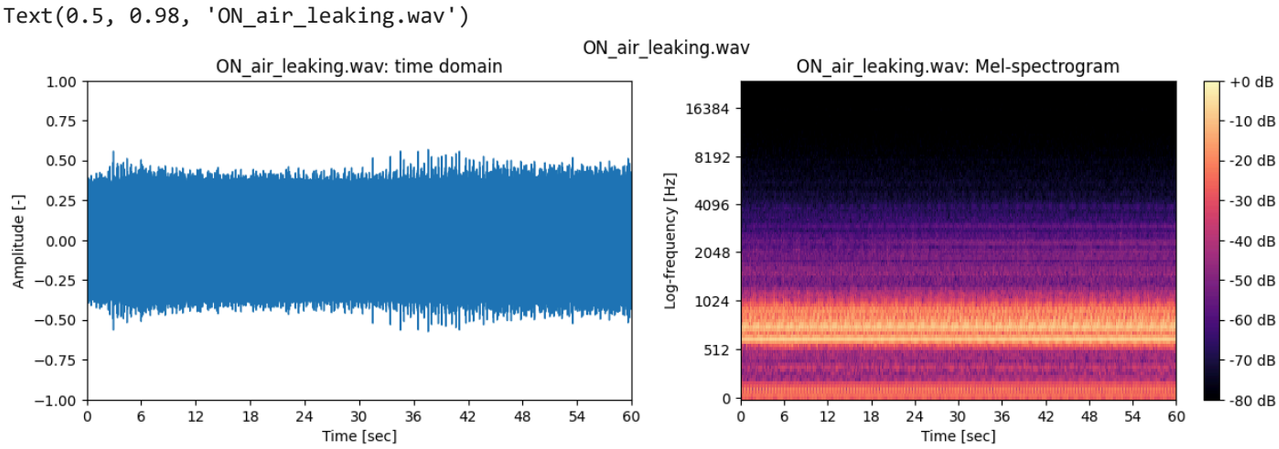
```
# This example is for plotting ON/leaking

y_vacuum, sr = librosa.load(ON_vacuum, sr=48000) # y_vacuum is audio data in on/vacuuming state and s
M_vacuum = librosa.feature.melspectrogram(y=y_vacuum, sr=sr, n_fft=N_FFT, hop_length=int(N_FFT/4), w

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))
librosa.display.waveshow(y_vacuum, sr=sr, ax=ax[0]) # time domain (raw data) plot
img = librosa.display.specshow(librosa.power_to_db(2*abs(M_vacuum)/N_FFT, ref=1), ax=ax[1], x_axis='
ax[0].set(title=ON_vacuum+": time domain", ylim=[-1, 1], xlim=[0, 60], ylabel="Amplitude [-]", xlabe
ax[1].set(title=ON_vacuum+": Mel-spectrogram", ylabel="Log-frequency [Hz]", xlabel="Time [sec]")
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_vacuum)
```

```
Text(0.5, 0.98, 'ON_vacuum.wav')
```




```
# This example is for plotting ON/leaking

y_airleaking, sr = librosa.load(ON_air_leaking, sr=48000) # y_vacuum is audio data in on/vacuuming st
M_airleaking = librosa.feature.melspectrogram(y=y_airleaking, sr=sr, n_fft=N_FFT, hop_length=int(N_FI

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))
librosa.display.waveshow(y_airleaking, sr=sr, ax=ax[0]) # time domain (raw data) plot
img = librosa.display.specshow(librosa.power_to_db(2*abs(M_airleaking)/N_FFT, ref=1), ax=ax[1], x_ax
ax[0].set(title=ON_air_leaking+": time domain", ylim=[-1, 1], xlim=[0, 60], ylabel="Amplitude [-]", 
ax[1].set(title=ON_air_leaking+": Mel-spectrogram", ylabel="Log-frequency [Hz]", xlabel="Time [sec]"
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_air_leaking)
```

```
Text(0.5, 0.98, 'ON_air_leaking.wav')
```



Task 2.2

Using the code blocks above, plot other data below.

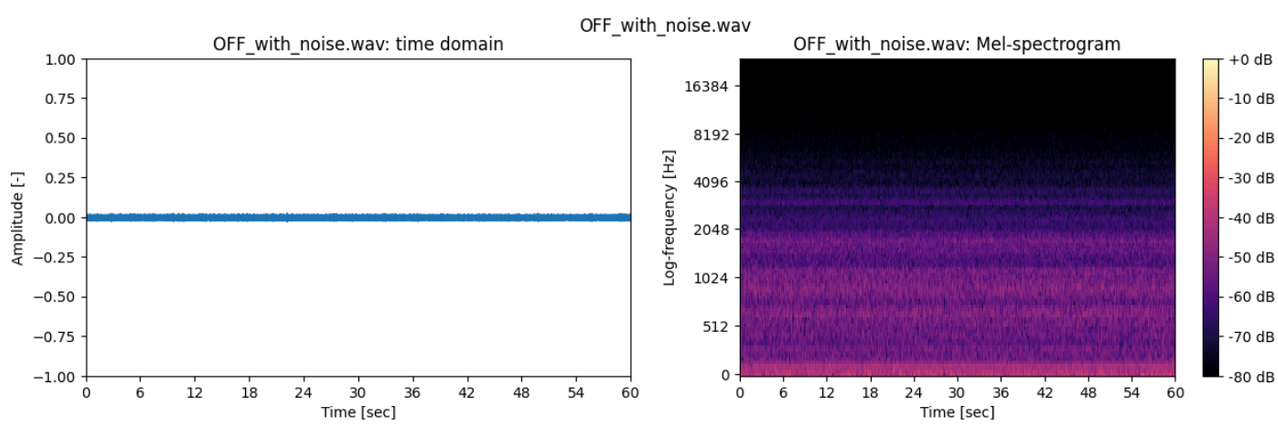
- OFF_with_noise.wav
- ON_vacuum_with_noise.wav
- ON_air_leaking_with_noise_wave.wav

By comparing and analyzing data, discuss differences between the states.

```
## variables for the following session and this task

y_off_noise, sr = librosa.load(OFF_with_noise, sr=48000) # y_off is audio data in off state and sr is
M_off_noise = librosa.feature.melspectrogram(y=y_off_noise, sr=sr, n_fft=N_FFT, hop_length=int(N_FFT,
```

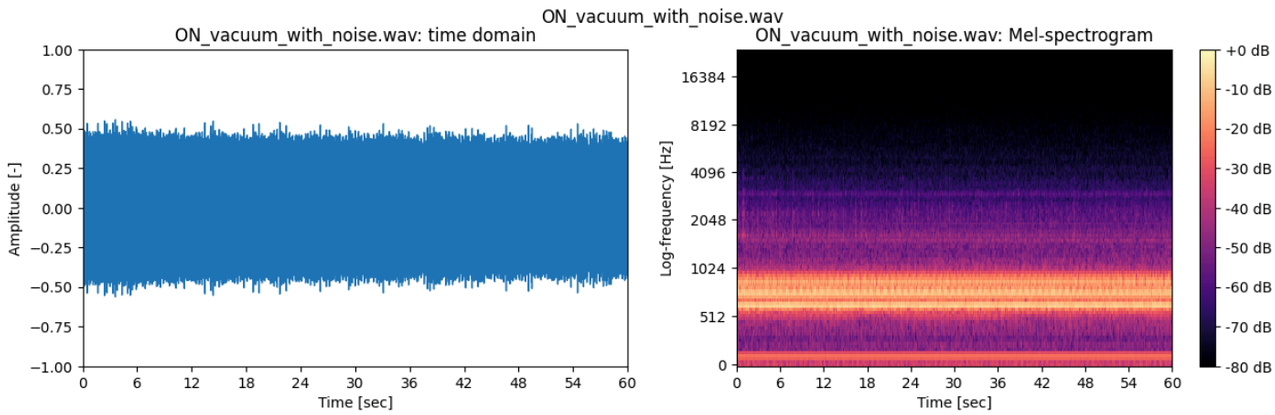
```
y_vacuum_noise, sr = librosa.load(ON_vacuum_with_noise, sr=48000) # y_vacuum is audio data in on/vacu
M_vacuum_noise = librosa.feature.melspectrogram(y=y_vacuum_noise, sr=sr, n_fft=N_FFT, hop_length=int

y_airleaking_noise, sr = librosa.load(ON_air_leaking_with_noise, sr=48000) # y_vacuum is audio data :
M_airleaking_noise = librosa.feature.melspectrogram(y=y_airleaking_noise, sr=sr, n_fft=N_FFT, hop_le
```

```
y_off_noise, sr = librosa.load(OFF_with_noise, sr=48000)

M_off_noise = librosa.feature.melspectrogram(
    y=y_off_noise, sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT/4),
    win_length=N_FFT,
    window='hann',
    n_mels=128
)

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))

librosa.display.waveshow(y_off_noise, sr=sr, ax=ax[0])

img = librosa.display.specshow(
    librosa.power_to_db(2*abs(M_off_noise)/N_FFT, ref=1),
    ax=ax[1], x_axis='time', y_axis='mel',
    sr=sr, vmin=-80, vmax=0
)

ax[0].set(title=OFF_with_noise + ": time domain",
          ylim=[-1, 1], xlim=[0, 60],
          ylabel="Amplitude [-]", xlabel="Time [sec]")

ax[1].set(title=OFF_with_noise + ": Mel-spectrogram",
          ylabel="Log-frequency [Hz]", xlabel="Time [sec]")

fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(OFF_with_noise)
plt.show()
```

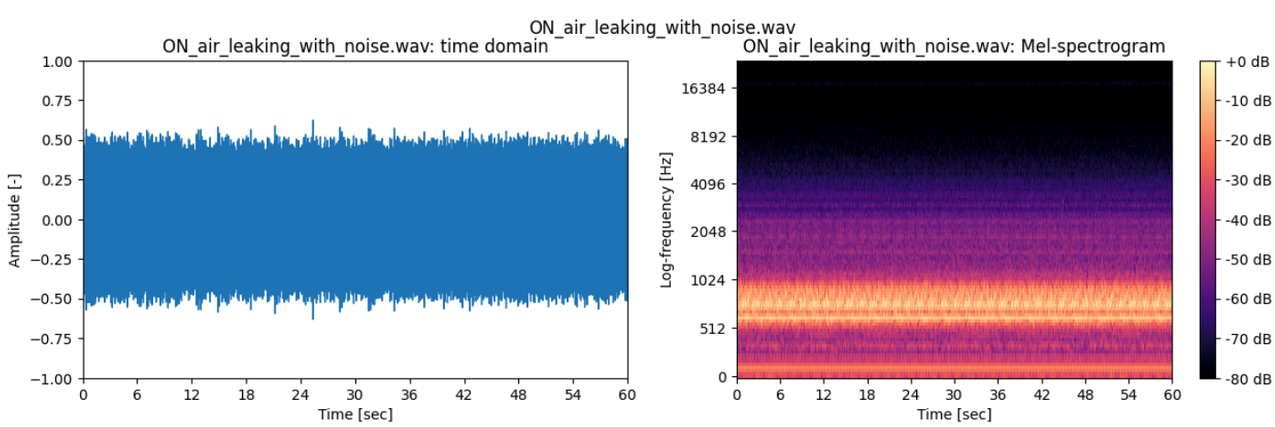



```
y_vacuum_noise, sr = librosa.load(ON_vacuum_with_noise, sr=48000)

M_vacuum_noise = librosa.feature.melspectrogram(
    y=y_vacuum_noise, sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT/4),
    win_length=N_FFT,
    window='hann',
    n_mels=128
)

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))

librosa.display.waveshow(y_vacuum_noise, sr=sr, ax=ax[0])

img = librosa.display.specshow(
    librosa.power_to_db(2*abs(M_vacuum_noise)/N_FFT, ref=1),
    ax=ax[1], x_axis='time', y_axis='mel',
    sr=sr, vmin=-80, vmax=0
)
```

```
ax[0].set(title=ON_vacuum_with_noise + ": time domain",
          ylim=[-1, 1], xlim=[0, 60],
          ylabel="Amplitude [-]", xlabel="Time [sec]")

ax[1].set(title=ON_vacuum_with_noise + ": Mel-spectrogram",
          ylabel="Log-frequency [Hz]", xlabel="Time [sec]")

fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_vacuum_with_noise)
plt.show()
```




```
y_airleaking_noise, sr = librosa.load(ON_air_leaking_with_noise, sr=48000)

M_airleaking_noise = librosa.feature.melspectrogram(
    y=y_airleaking_noise, sr=sr,
    n_fft=N_FFT,
    hop_length=int(N_FFT/4),
    win_length=N_FFT,
    window='hann',
    n_mels=128
)

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))

librosa.display.waveshow(y_airleaking_noise, sr=sr, ax=ax[0])

img = librosa.display.specshow(
    librosa.power_to_db(2*abs(M_airleaking_noise)/N_FFT, ref=1),
    ax=ax[1], x_axis='time', y_axis='mel',
    sr=sr, vmin=-80, vmax=0
)

ax[0].set(title=ON_air_leaking_with_noise + ": time domain",
          ylim=[-1, 1], xlim=[0, 60],
          ylabel="Amplitude [-]", xlabel="Time [sec]")

ax[1].set(title=ON_air_leaking_with_noise + ": Mel-spectrogram",
          ylabel="Log-frequency [Hz]", xlabel="Time [sec]")

fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_air_leaking_with_noise)
plt.show()
```



Feature	OFF with Noise	ON Vacuum with Noise	ON Air Leaking with Noise
Time-domain Amplitude	Very low, nearly flat	High and stable	High but fluctuating
Signal Pattern	Random noise	Smooth and continuous	Irregular and unstable
Energy Level	Low	High	High
Frequency Content	Spread randomly (no structure)	Concentrated in low–mid frequencies (500–1500 Hz)	Spread to higher frequencies
Spectrogram Pattern	Weak, scattered	Strong, consistent band	Strong but less uniform
Stability	No pattern	Stable operation	Unstable (turbulence)
Sound Type	Background noise	Smooth humming	Hissing / airflow
Key Identification	No machine activity	Steady machine running	Air leakage present

Labeling and CNN model data input pipeline

Let's make the input data with labeling (output) for the CNN model training. Before discretizing data for the inputs, let's understand how we split data. First we set the size of window (window size), so-called input data length. And then scan the entire dataset with a scanning interval as shown in Figure 9.

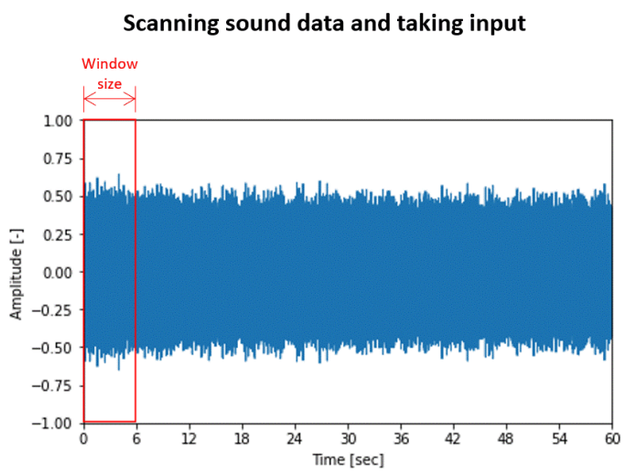


Figure 9 Scanning for taking input data

As an example, let's have the input data 1 second. However, when it comes to the MTConnect straem, the chunk size (minimum data length in a sequence) is 2048 data points, which means the chunk is 0.042667 sec because the sampling rate is 48000 Hz. We calculate how many chunks (N_{seq}) are close to 1 second. The number of chunks close to 1 second is 23 (actually 0.98133 sec). The number of data points for the window size is 47,104 which can be calculated by multiplying the chunk size and the number of the sequences ($N_{chunk} \times N_{seq}$). Also, let's have a scanning interval to equal the chunk size. That means in every interval size, 1-second data is taken as input data after performing signal processing. The signal processing (feature extraction) is the same as in the section above, Mel-spectrogram. Input data specifications and feature extraction are summarized below.

- Window size: 0.981333 sec
- N_{chunk} = 2048
- N_{seq} = 23
- f_s (sampling frequency) = 48000 Hz
- t_{window} = 0.98133 sec. ($N_{chunk} \times N_{seq} / f_s$)
- Scanning interval = 0.042667 sec (= 2048 data points)
- Feature: Mel-spectrogram
- N_FFT (number of FFT points) = 2048
- hop_length = 512
- Windowing function = hanning
- n_mels (number of Mel-filter banks) = 128

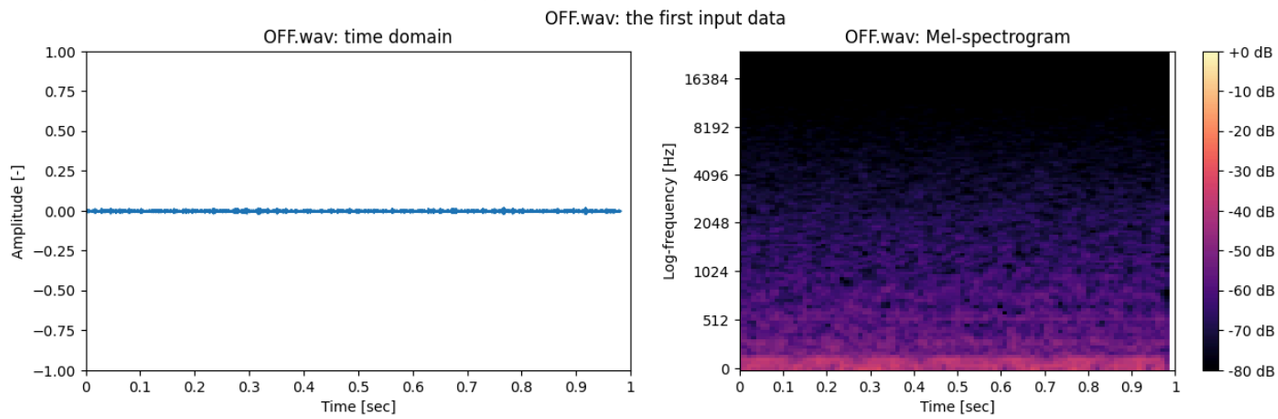
It is expected that the input data has a 2D-array and the shape (dimension) is (128×93). With this information, let's perform splitting data by scanning the dataset for generating the machine learning input and labeling.

```
# This code prints out length of the audio file (OFF.WAV) in second unit.
print("Total data points of an audio file: ", len(y_off))
print("Total length of an audio file: ",len(y_off)/sr, "sec")

n_chunk = 2048 # Number of data points in a chunk
n_seq = 23 # Number of the required sequences for a window
n_interval = 2048 # Number of data points for interval
n_total_lengh = len(y_off) # Number of total data points
n_window_length = n_chunk * n_seq # number of data points in a window
N_window = int((len(y_off) - n_window_length)/n_interval) + 1 # Number of windows from the data

print("\nTotal number of windows for training data input per file:", N_window)
print("Length of each window:",n_window_length/sr, "sec")
```

```
Total data points of an audio file:  2880000
Total length of an audio file:  60.0 sec

Total number of windows for training data input per file: 1384
Length of each window: 0.9813333333333333 sec
```

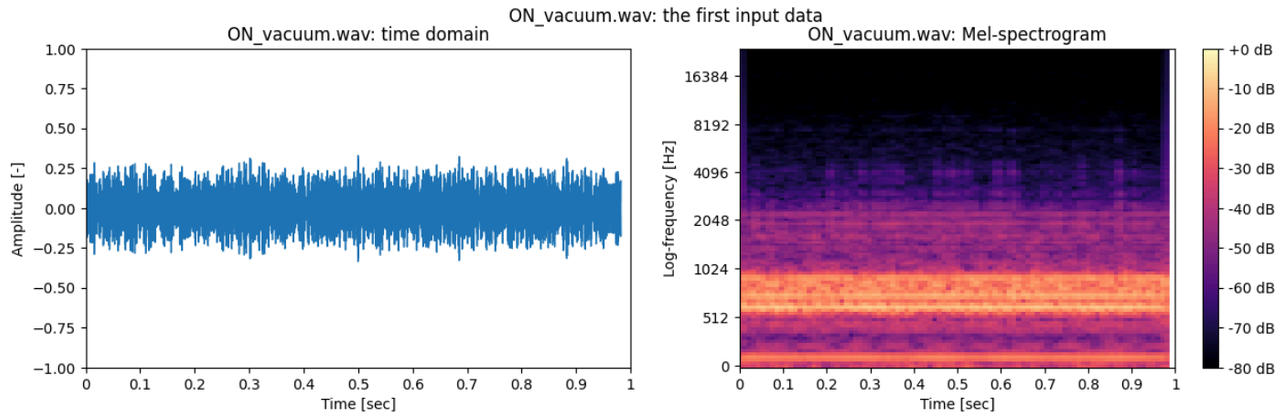
```
# Let's conduct feature extraction and visualization for the first data of OFF.wav
y_off_1 = y_off[0:n_chunk*n_seq] # taking the first window by indexing
M_off_1 = librosa.feature.melspectrogram(y=y_off_1, sr=sr, n_fft=N_FFT, hop_length=int(N_FFT/4), win_
S_off_1 = 2*abs(M_off_1)/N_FFT # power of Mel-spectrogram as input

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))
librosa.display.waveshow(y_off_1, sr=sr, ax=ax[0]) # time domain (raw data) plot
img = librosa.display.specshow(librosa.power_to_db(S_off_1, ref=1), ax=ax[1], x_axis='time', y_axis=
ax[0].set(title=OFF+": time domain", ylim=[-1, 1], xlim=[0, 1], ylabel="Amplitude [-]", xlabel="Time
ax[1].set(title=OFF+": Mel-spectrogram", ylabel="Log-frequency [Hz]", xlabel="Time [sec]")
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(OFF+": the first input data")
```

```
Text(0.5, 0.98, 'OFF.wav: the first input data')
```



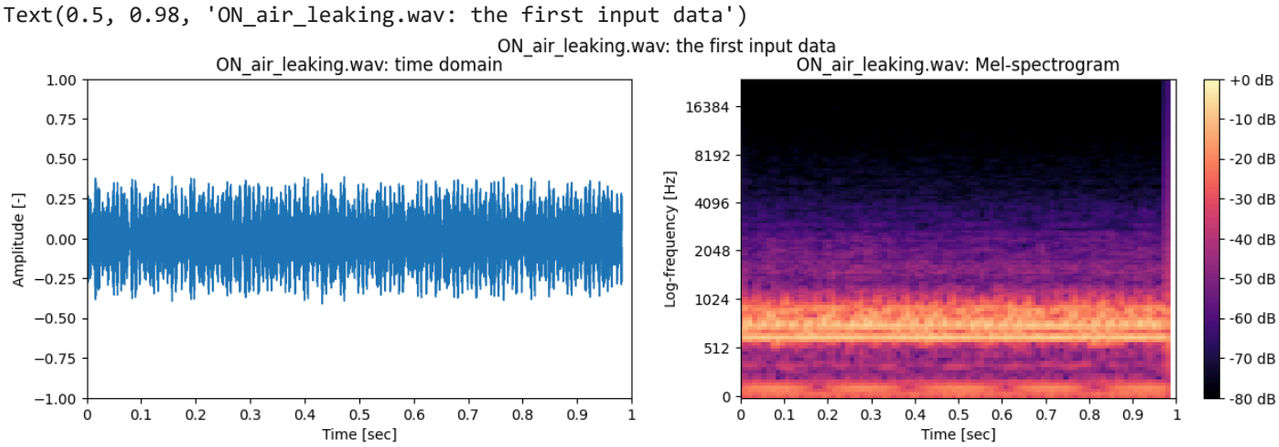

```
# Let's conduct feature extraction and visualization for the first data of ON_vacuuming.wav
y_vacuum_1 = y_vacuum[0:n_chunk*n_seq] # taking the first window by indexing
M_vacuum_1 = librosa.feature.melspectrogram(y=y_vacuum_1, sr=sr, n_fft=N_FFT, hop_length=int(N_FFT/4
S_vacuum_1 = 2*abs(M_vacuum_1)/N_FFT # power of Mel-spectrogram as input

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))
librosa.display.waveshow(y_vacuum_1, sr=sr, ax=ax[0]) # time domain (raw data) plot
img = librosa.display.specshow(librosa.power_to_db(S_vacuum_1, ref=1), ax=ax[1], x_axis='time', y_ax
ax[0].set(title=ON_vacuum+": time domain", ylim=[-1, 1], xlim=[0, 1], ylabel="Amplitude [-]", xlabel
ax[1].set(title=ON_vacuum+": Mel-spectrogram", ylabel="Log-frequency [Hz]", xlabel="Time [sec]")
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_vacuum+": the first input data")
```

```
Text(0.5, 0.98, 'ON_vacuum.wav: the first input data')
```




```
# Let's conduct feature extraction and visualization for the first data of ON_air_leaking.wav
y_airleaking_1 = y_airleaking[0:n_chunk*n_seq] # taking the first window by indexing
M_airleaking_1 = librosa.feature.melspectrogram(y=y_airleaking_1, sr=sr, n_fft=N_FFT, hop_length=int
S_airleaking_1 = 2*abs(M_airleaking_1)/N_FFT # power of Mel-spectrogram as input

fig, ax = plt.subplots(1,2, sharex=True, figsize=(15,4))
librosa.display.waveshow(y_airleaking_1, sr=sr, ax=ax[0]) # time domain (raw data) plot
img = librosa.display.specshow(librosa.power_to_db(S_airleaking_1, ref=1), ax=ax[1], x_axis='time', 
ax[0].set(title=ON_air_leaking+": time domain", ylim=[-1, 1], xlim=[0, 1], ylabel="Amplitude [-]", x
ax[1].set(title=ON_air_leaking+": Mel-spectrogram", ylabel="Log-frequency [Hz]", xlabel="Time [sec]"
cbar = fig.colorbar(img, ax=ax[1], format="%+2.0f dB")
fig.suptitle(ON_air_leaking+": the first input data")
```

```
Text(0.5, 0.98, 'ON_air_leaking.wav: the first input data')
```



Task 2.3

The first input data processing from an audio file is given above, can you take the second, and third input data? (*Hint: use indexing of the raw data (y)*)

How to generalize/express the process for n^{th} data input?

Generalization for n-th Input Data

Each input is extracted using a sliding window over the raw signal.

Let:

- n_interval = scanning interval (2048 samples)
- n_chunk × n_seq = window length (47104 samples)

Then, the n-th input is given by:

y_n = y[(n-1)*n_interval : (n-1)*n_interval + n_chunk*n_seq]

```
# Function to extract nth input window
def get_nth_input(y, n, n_interval, n_chunk, n_seq):
    start = (n - 1) * n_interval
    end = start + (n_chunk * n_seq)
    return y[start:end]
```

```
y_1 = get_nth_input(y_off, 1, n_interval, n_chunk, n_seq)
y_2 = get_nth_input(y_off, 2, n_interval, n_chunk, n_seq)
y_3 = get_nth_input(y_off, 3, n_interval, n_chunk, n_seq)
```

Now, let's build up the input pipeline for training model. The given function below, *get_input_data*, is to process feature extraction for making the training dataset.

```
def get_input_data(y, label, N_window, N_interval, N_seq, N_chunk, n_fft, hop_length, sr):
  ## Args decription below
  # y is a raw data (sound signal) for being processed.
  # label is the state (condition), string dtype
  # N_window is number of windows to be splitted for input data from the raw data.
  # N_interval is the scanning interval.
  # N_seq is number of the seuqneces in MTConnect stream to have a specific length of window.
  # N_chunk is a chunk size in which a sequence of MTConnect has data points.
  # n_fft is number of FFT points.
  # hop_length is hop length when spectrogram calculation
  # sr is sampling rate of the audio.

  X = [] # initialize input data array
  Y = [] # initialize label data array
  for i in range(N_window):
    y_temp = y[i*N_interval:(i*N_interval)+N_chunk*N_seq]
    x_temp = librosa.feature.melspectrogram(y=y_temp, sr=sr, n_fft=n_fft,
                                            hop_length=hop_length,
                                            win_length=n_fft, window='hann',
                                            n_mels=128)
    feature_temp = 2*abs(x_temp)/n_fft
    Y.append(label)
    X.append(feature_temp)

  return np.array(X), np.array(Y)
```

```
# X_off and Y_off are training input data and labels, respectively.
# It takes tens of seconds. If you create your own function to split data by taking not raw data but
X_off, Y_off = get_input_data(y=y_off, label="OFF", N_window=N_window, N_interval=n_interval, N_seq=n

print("X_off shape is",X_off.shape)
print("Y_off shape is",Y_off.shape)
```

```
X_off shape is (1384, 128, 93)
Y_off shape is (1384,)
```

Let's understand the input data after processing (Figure 10). X_off (training input) has (k, n, m) shape, (1384, 128, 93) in this case. Y_off (training label) has (1384,) shape. This means the number of data for training for "OFF" label is 1384. Each input data (window) has 128×93 size of 2D-array, which is the power of the Mel-spectrogram. The column, n=128, represents frequency (Mel) components and the row, m=93, represents time components. Because raw sound data itself in WAV format is normalized from -1 to 1, we don't need to normalize them for training input.

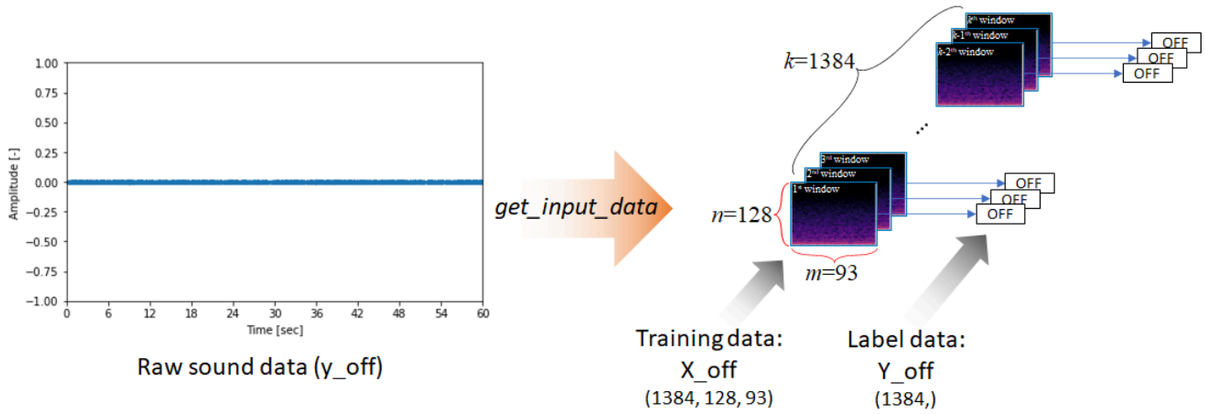
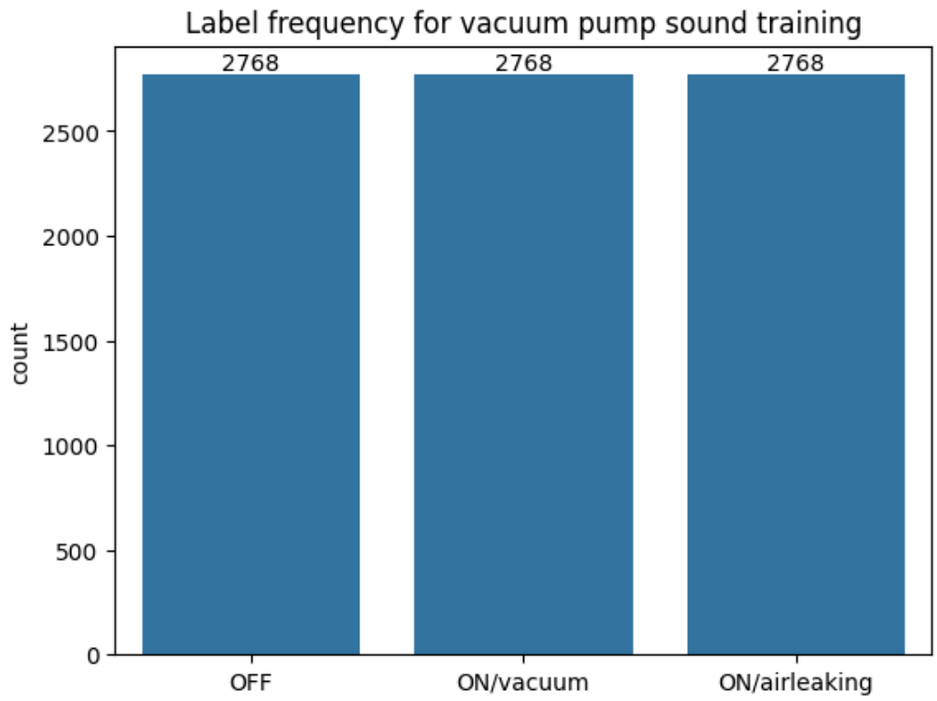


Figure 10 Understanding processing and shape of input data for training

```
### keep working on other data, this may take a few minutes

X_off_noise, Y_off_noise = get_input_data(y=y_off_noise, label="OFF", N_window=N_window, N_interval=n

X_vacuum, Y_vacuum = get_input_data(y=y_vacuum, label="ON/vacuum", N_window=N_window, N_interval=n_in
X_vacuum_noise, Y_vacuum_noise = get_input_data(y=y_vacuum_noise, label="ON/vacuum", N_window=N_windo

X_airleaking, Y_airleaking = get_input_data(y=y_airleaking, label="ON/airleaking", N_window=N_window,
X_airleaking_noise, Y_airleaking_noise = get_input_data(y=y_airleaking_noise, label="ON/airleaking",
```

```
# concatenate all data in training data
# number of each input dataset is 1348, and we have total 6 data which has same size
# therefore, number of training data will be 8088
X_data = np.concatenate((X_off, X_off_noise, X_vacuum, X_vacuum_noise, X_airleaking, X_airleaking_no
Y_data = np.concatenate((Y_off, Y_off_noise, Y_vacuum, Y_vacuum_noise, Y_airleaking, Y_airleaking_no
```

```
import seaborn as sns
# seaborn is a statictical data visualization Python package

# before we plot the label frequency, we can expect how much data is in each label.
# Calculate the frequencies before you run this code.
# Are the results as you expected?

plt.figure()
plt.title("Label frequency for vacuum pump sound training")
ax = sns.countplot(x=Y_data)
for container in ax.containers:
  ax.bar_label(container)
plt.show()
```



To handling and evaluate training data, we will use [scikit-learn Python package]() which is known as sklearn. Practically, [sklearn](), [keras]() with [TensorFlow]() are most widely used tool for ML training and implementation.

Before we split train and test (validation) dataset, let's encode the label data for training format. As defined, our problem (purpose) is a multi-class classification. Each class (normally formatted as positive integer) represents a state (operational condition) in this example.

```
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.utils import to_categorical

enc = LabelEncoder() # encoder object
enc.fit(Y_data)
Y_data_enc = enc.transform(Y_data) # Encoding from string labels to integers; 0: OFF, 1: ON/airleaki
Y_label = to_categorical(Y_data_enc) # categorical labeling of the integer index; [1 0 0]: OFF, [0 1
```

```
# using this code block,
# check the encoding performed as you expected.
# Note that, because order of encoding is sorted by ascending order,
# 0 represents "OFF", 1 represents "ON/airleaking", 2 represents "ON/vacuum"
# The encoded integer index means principle axis of the categorical matrix,
# for the model training, 0, 1, and 2 turn out [1 0 0], [0 1 0], and [0 0 1], respectively.

print("Y_data:", Y_data)
print("Encoded Y_data:", Y_data_enc)
print("Encoded classes:", enc.classes_)
print("Categorical transform:", Y_label)
```

```
Y_data: ['OFF' 'OFF' 'OFF' ... 'ON/airleaking' 'ON/airleaking' 'ON/airleaking']
Encoded Y_data: [0 0 0 ... 1 1 1]
Encoded classes: ['OFF' 'ON/airleaking' 'ON/vacuum']
Categorical transform: [[1. 0. 0.]
 [1. 0. 0.]
 [1. 0. 0.]
 ...
 [0. 1. 0.]
 [0. 1. 0.]
 [0. 1. 0.]]
```

```
# Let's split training and testing dataset.
# X_train and Y_train will be used for training weights, bias, filters and so on.
# X_test and Y_test will be used for validating while training.
# In this example, we have 20% test data size. Therefore, the training data is 80%.

from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X_data, Y_label, test_size=0.2, random_state=40)
print("Number of data for training: {}".format(len(X_train)))
print("Number of data for validation: {}".format(len(X_test)))
```

```
Number of data for training: 6643
Number of data for validation: 1661
```

Now, we are ready to start training CNN model.

2.2 CNN model training

For the operational state prediction of the vacuum pump, we will use a convolutional neural network (CNN or ConvNet) architecture. While CNNs are mostly used for image classification/recognition, they also have been widely applied for sound recognition and classification tasks. CNNs are a type of multi-layer neural networks that typically contain convolution layers, subsampling layers, and fully connected (FC) layers. Even if the networks are complex because of a large amount of connectivity, the use of shared wieghts within layers helps in reducing the number of parameters that should be trained. Parameter sharing makes CNNs greatly reduce the number of unique model parameters and significantly increase the size of the network without requiring corresponding increase in training data.

A CNN can have a few or hundreds of layers that each learn to detect different features of an image. Filters are applied to each training image at different resolutions, and output of each convoluted image is used as the input of the next layer. Visit [Towards Data Science](), and [Analytics Vidhya]() to grasp more about CNN.

The CNN architecture in this example is shown in Figure 11. Red-colored parameters in the architecture are adjustable hyperparameters. We will use two convolutional layers and two hidden layers. If you are interested, try [common CNN archtectures](), such as AlexNet, VGG, GoogleNet, and so on. As input, the CNN takes tensors of shape (freq. components, time components) of Mel-spectrogram. The output node will be one among "OFF" (0), "ON/airleaking" (1), and "ON/vacuum" (2). Let's create the CNN architecture using TensorFlow and Keras API.

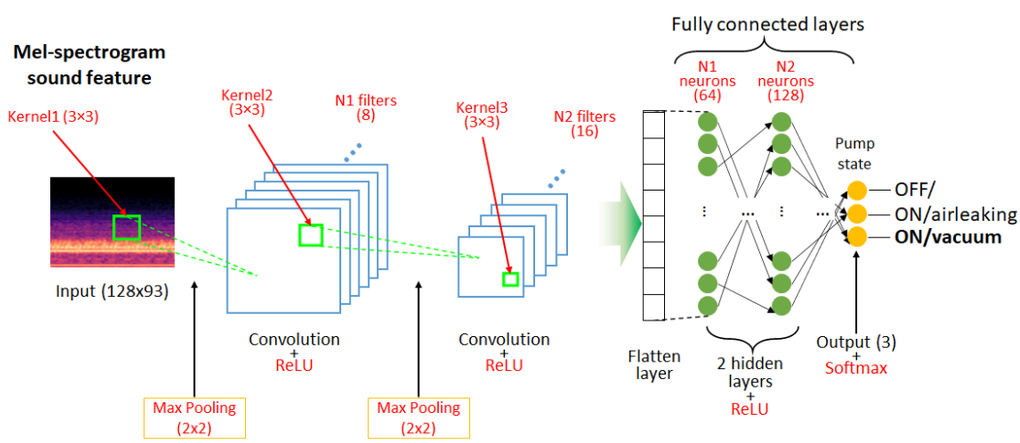


Figure 10 CNN architecture for vacuum pump operational state classification

Creating CNN model

```
import tensorflow as tf
```

```
from tensorflow.keras import layers

model = tf.keras.Sequential() # model obj. takes sequential model
model.add(layers.Reshape((X_train.shape[1], X_train.shape[2], 1), input_shape=(X_train.shape[1], X_tr
model.add(layers.Conv2D(filters=8, kernel_size=(3,3), activation='relu')) # first 2D CNN layer
model.add(layers.MaxPooling2D(pool_size=(3,3))) # max pooling layer
model.add(layers.Conv2D(filters=16, kernel_size=(3,3), activation='relu')) # second 2D CNN layer
model.add(layers.MaxPooling2D(pool_size=(3,3))) # max pooling layer
model.add(layers.Flatten()) # flatten layer
model.add(layers.Dense(64, activation='relu')) # first hidden layer
model.add(layers.Dense(128, activation='relu')) # second hidden layer
model.add(layers.Dense(units=Y_train.shape[1], activation='softmax', name='output')) # output layer
model.summary() # This will show the model object
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/reshape.py:38: UserWarning: Do not
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
reshape (Reshape)	(None, 128, 93, 1)	0
conv2d (Conv2D)	(None, 126, 91, 8)	80
max_pooling2d (MaxPooling2D)	(None, 42, 30, 8)	0
conv2d_1 (Conv2D)	(None, 40, 28, 16)	1,168
max_pooling2d_1 (MaxPooling2D)	(None, 13, 9, 16)	0
flatten (Flatten)	(None, 1872)	0
dense (Dense)	(None, 64)	119,872
dense_1 (Dense)	(None, 128)	8,320
output (Dense)	(None, 3)	387

Total params: 129,827 (507.14 KB)
Trainable params: 129,827 (507.14 KB)
Non-trainable params: 0 (0.00 B)

Compiling and training the model

By running the code block below, let's compile the CNN model and then train it!

```
## Other hyperparameters
BATCH = 256 # the number of samples per gradient update
EPOCH = 5 # how many cycles will be trained in an entire dataset passed forward and backward through
LR = 10e-4 # learning rate is to control how much to change the model in response to the estimated er

# compiling the CNN model, loss function is categorical crossentropy and training metric is accuracy
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=LR), loss='categorical_crossentropy',

# This will start model training and take a few minutes
history = model.fit(X_train, Y_train, validation_data=(X_test, Y_test), batch_size=BATCH, epochs=EPO(
```

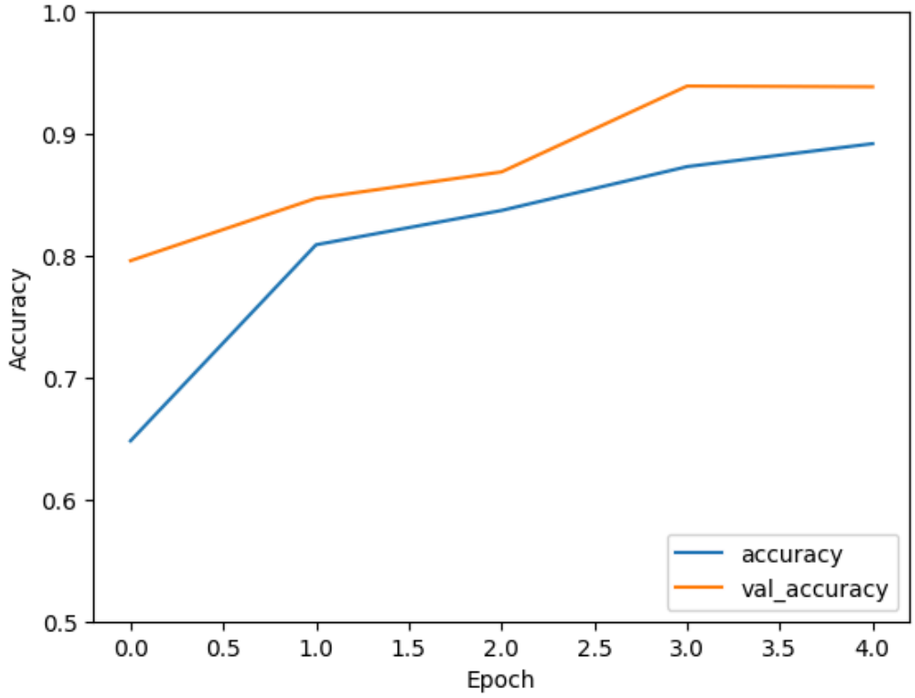
```
Epoch 1/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 31s 1s/step - accuracy: 0.6481 - loss: 0.9543 - val_accuracy: 0.7959 - val_
Epoch 2/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 28s 1s/step - accuracy: 0.8090 - loss: 0.3818 - val_accuracy: 0.8471 - val_
Epoch 3/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 29s 1s/step - accuracy: 0.8371 - loss: 0.2758 - val_accuracy: 0.8688 - val_
Epoch 4/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 40s 1s/step - accuracy: 0.8731 - loss: 0.2356 - val_accuracy: 0.9392 - val_
Epoch 5/5
26/26 ━━━━━━━━━━━━━━━━━━━━ 29s 1s/step - accuracy: 0.8919 - loss: 0.2043 - val_accuracy: 0.9386 - val_
```

Evaluate model

Let's plot the training history and perform basic evaluation of the trained model on the test dataset.

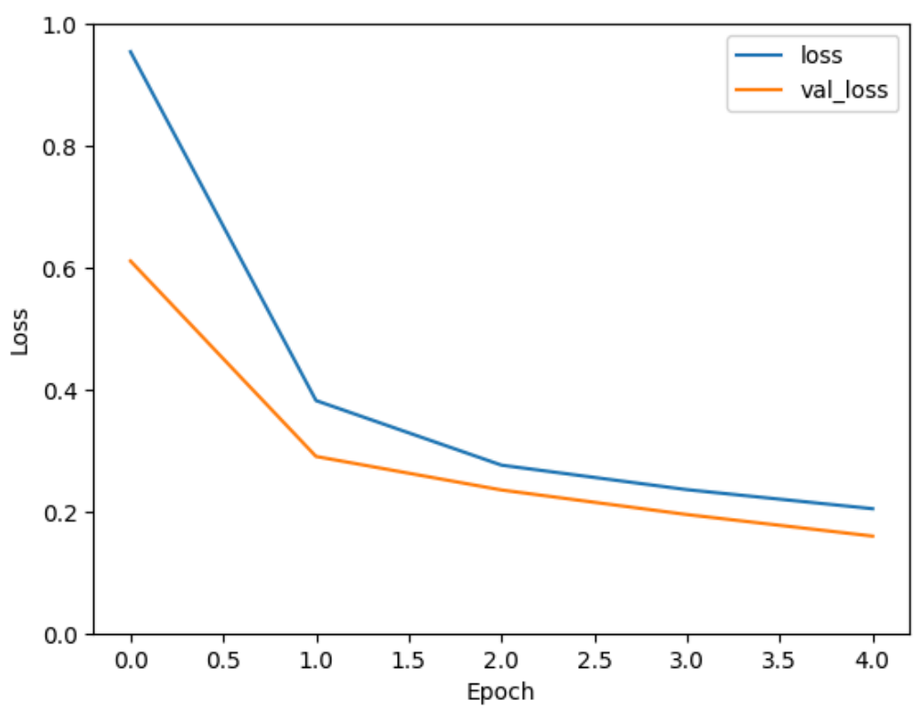
```
# This is for accuracy in training history
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label ='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
<matplotlib.legend.Legend at 0x7bb2b1da59d0>
```




```
# This is for loss in training history
plt.plot(history.history['loss'], label='loss')
plt.plot(history.history['val_loss'], label ='val_loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.ylim([0, 1])
plt.legend(loc='upper right')

test_loss, test_acc = model.evaluate(X_test, Y_test, verbose=2)
```

```
52/52 - 4s - 84ms/step - accuracy: 0.9386 - loss: 0.1594
```



Saving the trained model

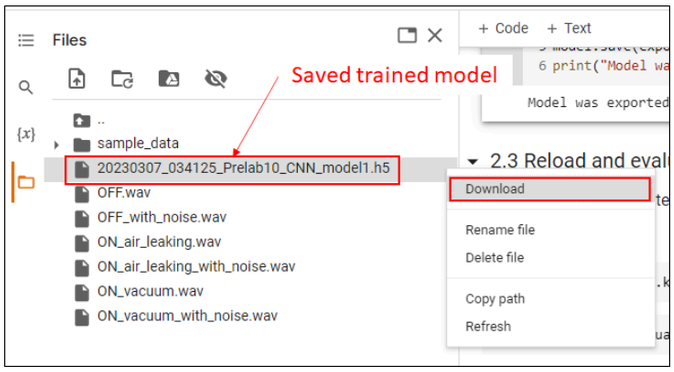
Then, let's save the trained model to implement it on Raspberry Pi. In this case we will use another standard binary format, [Hierarchical Data Format](#) so-called HDF5 or H5. Because Keras supports HDF5 format, we can take advantage of it.

```
import datetime

t = datetime.datetime.now().strftime("%Y%m%d_%H%M%S") # generate date time string to identify the mo
export_name = t+"_Prelab10_CNN_model1.h5" # this is eventually filename.
model.save(export_name) # save method save the model with the file name
print("Model was exported in this path: '{}'".format(export_name))
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(
Model was exported in this path: '20260407_173100_Prelab10_CNN_model1.h5'
```

2.3 Reload and evaluate the CNN model

Let's reload and evaluate the saved model. Before that, **download the model to your laptop from the Colab Files session as the capture below to use in the lab activity.**



Before we move on to reload and evaluate the model, let's figure out the evaluation means. For the machine learning model evaluation, we test it using various methods and performance metrics. We will check the confusion matrix, and some performance metrics (accuracy, precision, recall, and F1 score). Go through [this article]() and perform Task 2.4.

Task 2.4

1. What is confusion matrix? Why is it used for evaluation?
2. Explain the performance metrics. * Accuracy * Precision * Recall * F1 score
3. Which performance metric above is most important for this problem and why?

📊 Confusion Matrix and Performance Metrics

🔹 What is a Confusion Matrix?

A **confusion matrix** is a table used to evaluate a classification model by comparing **actual labels** with **predicted labels**.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

🔹 Why is it used?

- Helps understand **model performance in detail**
- Shows **types of errors**:
  - False Positive (FP)
  - False Negative (FN)
- More informative than accuracy alone
- Important for **imbalanced datasets**

Double-click (or enter) to edit

📈 Performance Metrics

✅ 1. Accuracy

Definition: Overall correctness of the model

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

✓ Works well for balanced data
❌ Misleading for imbalanced data

✅ 2. Precision

Definition: How many predicted positives are correct

$$Precision = \frac{TP}{TP + FP}$$

✓ Important when **false alarms must be minimized**

✅ 3. Recall (Sensitivity)

Definition: How many actual positives are correctly detected

$$Recall = \frac{TP}{TP + FN}$$

✓ Important when **missing true events is critical**

✅ 4. F1 Score

Definition: Balance between Precision and Recall

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

✓ Best for **imbalanced datasets**
✓ Considers both FP and FN

Most Important Metric for This Problem

For the **Air Compressor Sound Monitoring (Run vs Rest)** problem:

👉 **Recall is the most important**

Reason:

- Missing a **running condition (False Negative)** is critical
- It may lead to:
  - Undetected faults
  - System failure
  - Poor monitoring

Practical Choice:

👉 **F1 Score is preferred in practice**

✓ Because it balances:

- Detecting true events (Recall)
- Avoiding false alarms (Precision)

```
# if you want to load other trained model, you have to change the model filename, export_name in thi
reloaded = tf.keras.models.load_model(export_name)
```

```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_
```

```
# let's call the summary of the model to see it is loaded correctly.
reloaded.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
reshape (Reshape)	(None, 128, 93, 1)	0
conv2d (Conv2D)	(None, 126, 91, 8)	80
max_pooling2d (MaxPooling2D)	(None, 42, 30, 8)	0
conv2d_1 (Conv2D)	(None, 40, 28, 16)	1,168
max_pooling2d_1 (MaxPooling2D)	(None, 13, 9, 16)	0
flatten (Flatten)	(None, 1872)	0
dense (Dense)	(None, 64)	119,872
dense_1 (Dense)	(None, 128)	8,320
output (Dense)	(None, 3)	387

Total params: 129,829 (507.15 KB)
Trainable params: 129,827 (507.14 KB)

```
# evaluate method will evaluate the model using the given data
# in this example, we wil use the test dataset splitted from the training data
# the output will be the loss (cross-entropy) and accuracy (metrics when compiling the model)
reloaded.evaluate(X_test, Y_test)
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 3s 46ms/step - accuracy: 0.9386 - loss: 0.1594
[0.15938378870487213, 0.9385911822319031]
```

```
# predict method returns the result of the model by the given input data
# because we have 3 classes and the output node was trained by Softmax activation function,
# each data has 3 probabilities at the end
# sum of the 3 probabilities is 1.

Y_pred = reloaded.predict(X_test)
Y_pred
# is the result as you expected?
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 2s 46ms/step
array([[9.0979154e-07, 5.2215707e-01, 4.7784197e-01],
       [9.9794012e-01, 2.3899496e-05, 2.0358888e-03],
       [2.2300761e-10, 9.7896951e-01, 2.1030469e-02],
       ...,
       [9.9793637e-01, 2.3931405e-05, 2.0397087e-03],
       [1.5770918e-02, 5.6038471e-03, 9.7862524e-01],
       [9.9793637e-01, 2.3933024e-05, 2.0396872e-03]], dtype=float32)
```

```
# Let's check the true output, Y_test in this case.
# As we did above, the principle axis means the class.
Y_test
```

```
array([[0., 1., 0.],
       [1., 0., 0.],
       [0., 1., 0.],
       ...,
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.]])
```

```
# argmax method makes your prediction result to the label index
# among the 3 probabilities, element of th highest probability will be the principle axis for the lal
# for example, [0 1 0] will be 1.

# run this and see if the result is as expected.

Y_pred_ind = Y_pred.argmax(1)
print("Prediction label index:", Y_pred_ind)
```

```
Prediction label index: [1 0 1 ... 0 2 0]
```

```
# We do the same for the true Y (Y_test_ind)
```

```
Y_test_ind = Y_test.argmax(1)
print("True label index:", Y_test_ind)
```

```
True label index: [1 0 1 ... 0 2 0]
```

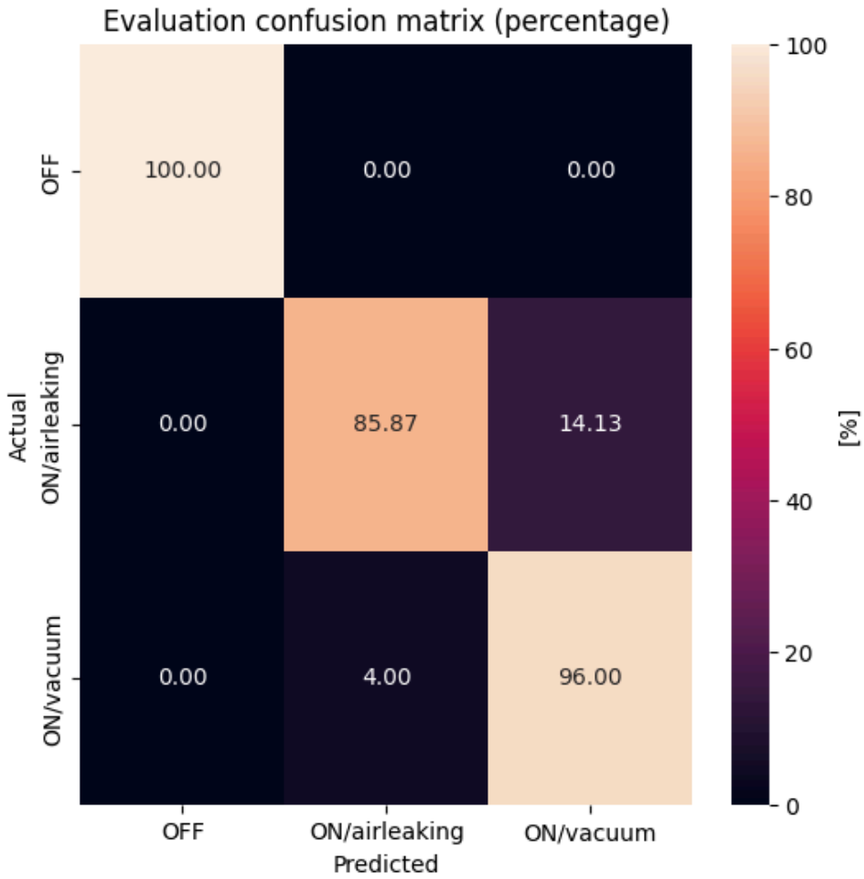
```
# using LabelEncoder object we defined,
# we decode the encoded index to the actual labels, 0:'OFF', 1:'ON/airleaking', 2:'ON/vacuum'

Y_test_label = enc.inverse_transform(Y_test_ind)
Y_pred_label = enc.inverse_transform(Y_pred_ind)
print("True labels:", Y_test_label)
print("Prediced labels:", Y_pred_label)
```

```
True labels: ['ON/airleaking' 'OFF' 'ON/airleaking' ... 'OFF' 'ON/vacuum' 'OFF']
Prediced labels: ['ON/airleaking' 'OFF' 'ON/airleaking' ... 'OFF' 'ON/vacuum' 'OFF']
```

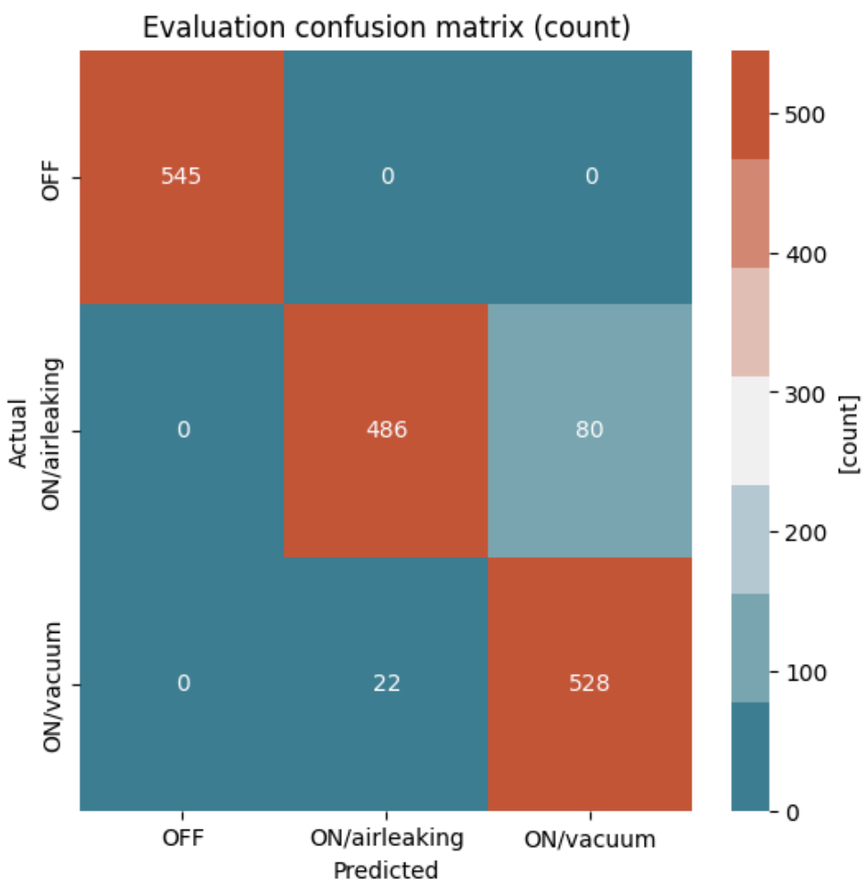
Basically, you performed basic evaluation (only loss and accuracy) and handling the prediction results. Let's try evaluating the model in detail using confusion matrix and other performance metrics.

```
# plot_cm function is for plotting confusion matrix

from sklearn.metrics import confusion_matrix

def plot_cm(y_true, y_pred, class_names, exp:str):
  fig, ax = plt.subplots(figsize=(6, 6))

  cm = confusion_matrix(y_true, y_pred)
  if exp == "percentage":
    cmn = cm.astype('float')/cm.sum(axis=1)[:, np.newaxis]*100
    ax = sns.heatmap(
      cmn,
      annot=True,
      fmt=".2f",
      ax=ax,
      cbar_kws={'label': '[%]'}
      )
  else:
    cmn = cm
    ax = sns.heatmap(
      cmn,
      annot=True,
      fmt="d",
      cmap=sns.diverging_palette(220, 20, n=7),
      ax=ax,
      cbar_kws={'label': '[{}]'.format(exp)}
      )

  plt.ylabel('Actual')
  plt.xlabel('Predicted')
  plt.title('Evaluation confusion matrix ({})'.format(exp))
  ax.set_xticklabels(class_names)
  ax.set_yticklabels(class_names)
  b, t = plt.ylim() # discover the values for bottom and top
  plt.ylim(b, t) # update the ylim(bottom, top) values
  plt.show()
```

```
# Evaluation: Confusion matrix, percentage
plot_cm(Y_test_ind, Y_pred_ind, enc.classes_, "percentage")
```




```
# Evaluation: Confusion matrix, count
plot_cm(Y_test_ind, Y_pred_ind, enc.classes_, "count")
```



```
# performance metrics in micro average
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

pred_accuracy = accuracy_score(Y_test_ind, Y_pred_ind)
pred_precision = precision_score(Y_test_ind, Y_pred_ind, average='macro')
pred_recall = recall_score(Y_test_ind, Y_pred_ind, average='macro')
pred_f1 = f1_score(Y_test_ind, Y_pred_ind, average='macro')

print("Accuracy: {:.2f}%".format(pred_accuracy*100))
print("Precision: {:.2f}%".format(pred_precision*100))
print("Recall: {:.2f}%".format(pred_recall*100))
print("F1-score: {:.2f}%".format(pred_f1*100))
```

```
Accuracy: 93.86%
Precision: 94.17%
Recall: 93.96%
F1-score: 93.90%
```

```
# performance metrics in each class

pred_precision_each = precision_score(Y_test_ind, Y_pred_ind, average=None)
pred_recall_each = recall_score(Y_test_ind, Y_pred_ind, average=None)
pred_f1_each = f1_score(Y_test_ind, Y_pred_ind, average=None)

precision_result_each = (enc.classes_[0], pred_precision_each[0]*100,
                         enc.classes_[1], pred_precision_each[1]*100,
                         enc.classes_[2], pred_precision_each[2]*100)
recall_result_each = (enc.classes_[0], pred_recall_each[0]*100,
                      enc.classes_[1], pred_recall_each[1]*100,
                      enc.classes_[2], pred_recall_each[2]*100,)
f1_result_each = (enc.classes_[0], pred_f1_each[0]*100,
                  enc.classes_[1], pred_f1_each[1]*100,
                  enc.classes_[2], pred_f1_each[2]*100,)

print("Precision each: {}={:.2f}%, {}={:.2f}%, {}={:.2f}%".format(*precision_result_each))
print("Recall each: {}={:.2f}%, {}={:.2f}%, {}={:.2f}%".format(*recall_result_each))
print("F1 each: {}={:.2f}%, {}={:.2f}%, {}={:.2f}%".format(*f1_result_each))
```

```
Precision each: OFF=100.00%, ON/airleaking=95.67%, ON/vacuum=86.84%
Recall each: OFF=100.00%, ON/airleaking=85.87%, ON/vacuum=96.00%
F1 each: OFF=100.00%, ON/airleaking=90.50%, ON/vacuum=91.19%
```

Task 2.5

Using the given examples above, try training different CNN architectures and select the best model. You may try changing number of CNN layers, number of hidden layers, hyperparameters, and so on. But you should take reasonable changes.

But, you can use the same feature (Mel-spectrogram, shape=(128,93)) above. Then **summarize your model (details of architecture), and evaluate it. Note that you need to use this model for next section and Lab10.**

```
## add more code blocks below as much as you need.

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matri
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Check input shape
print("X_train shape :", X_train.shape)
print("Y_train shape :", Y_train.shape)
print("X_test shape  :", X_test.shape)
print("Y_test shape  :", Y_test.shape)
print("Class labels  :", enc.classes_)
```

```
X_train shape : (6643, 128, 93)
Y_train shape : (6643, 3)
X_test shape  : (1661, 128, 93)
```

```
Y_test shape  : (1661, 3)
Class labels  : ['OFF' 'ON/airleaking' 'ON/vacuum']
```

```
# ==========================================
# Build CNN models
# ==========================================

input_shape = (X_train.shape[1], X_train.shape[2], 1)
num_classes = Y_train.shape[1]

def build_cnn_model(model_type="A"):
    model = models.Sequential()
    model.add(layers.Input(shape=(X_train.shape[1], X_train.shape[2])))
    model.add(layers.Reshape((X_train.shape[1], X_train.shape[2], 1)))

    if model_type == "A":
        # Baseline model
        model.add(layers.Conv2D(8, (3,3), activation='relu', padding='same'))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Conv2D(16, (3,3), activation='relu', padding='same'))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Flatten())
        model.add(layers.Dense(64, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))

    elif model_type == "B":
        # Slightly deeper model
        model.add(layers.Conv2D(16, (3,3), activation='relu', padding='same'))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same'))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Flatten())
        model.add(layers.Dense(128, activation='relu'))
        model.add(layers.Dense(64, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))

    elif model_type == "C":
        # Deeper + dropout model
        model.add(layers.Conv2D(16, (3,3), activation='relu', padding='same'))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Conv2D(32, (3,3), activation='relu', padding='same'))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Conv2D(64, (3,3), activation='relu', padding='same'))
        model.add(layers.MaxPooling2D((2,2)))
        model.add(layers.Flatten())
        model.add(layers.Dense(128, activation='relu'))
        model.add(layers.Dropout(0.3))
        model.add(layers.Dense(64, activation='relu'))
        model.add(layers.Dense(num_classes, activation='softmax'))

    return model
```

```
# ==========================================
# Train and evaluate models
# ==========================================

BATCH = 128
EPOCHS = 20
LR = 1e-3

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=4,
    restore_best_weights=True
)

results = []
histories = {}
trained_models = {}

for model_name in ["A", "B", "C"]:
    print(f"\nTraining Model {model_name} ...")

    model = build_cnn_model(model_name)
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=LR),
```

```
        optimizer=tf.keras.optimizers.Adam(learning_rate=LR),
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    history = model.fit(
        X_train, Y_train,
        validation_data=(X_test, Y_test),
        epochs=EPOCHS,
        batch_size=BATCH,
        verbose=1,
        callbacks=[early_stop]
    )

    # Prediction
    Y_pred = model.predict(X_test)
    Y_pred_ind = np.argmax(Y_pred, axis=1)
    Y_test_ind = np.argmax(Y_test, axis=1)

    acc = accuracy_score(Y_test_ind, Y_pred_ind)
    prec = precision_score(Y_test_ind, Y_pred_ind, average='macro')
    rec = recall_score(Y_test_ind, Y_pred_ind, average='macro')
    f1 = f1_score(Y_test_ind, Y_pred_ind, average='macro')

    results.append({
        "Model": model_name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1 Score": f1
    })

    histories[model_name] = history
    trained_models[model_name] = model
```

```
Epoch 3/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 43s 826ms/step - accuracy: 1.0000 - loss: 0.0086 - val_accuracy: 1.0000 -
Epoch 4/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 83s 840ms/step - accuracy: 1.0000 - loss: 0.0032 - val_accuracy: 1.0000 -
Epoch 5/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 80s 800ms/step - accuracy: 1.0000 - loss: 0.0018 - val_accuracy: 1.0000 -
Epoch 6/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 82s 808ms/step - accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 1.0000 -
Epoch 7/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 44s 850ms/step - accuracy: 1.0000 - loss: 7.7946e-04 - val_accuracy: 1.00
Epoch 8/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 81s 818ms/step - accuracy: 1.0000 - loss: 5.7254e-04 - val_accuracy: 1.00
Epoch 9/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 88s 956ms/step - accuracy: 1.0000 - loss: 4.5224e-04 - val_accuracy: 1.00
Epoch 10/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 75s 820ms/step - accuracy: 1.0000 - loss: 3.5546e-04 - val_accuracy: 1.00
Epoch 11/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 44s 851ms/step - accuracy: 1.0000 - loss: 2.8480e-04 - val_accuracy: 1.00
Epoch 12/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 81s 828ms/step - accuracy: 1.0000 - loss: 2.3552e-04 - val_accuracy: 1.00
Epoch 13/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 41s 798ms/step - accuracy: 1.0000 - loss: 1.9995e-04 - val_accuracy: 1.00
Epoch 14/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 82s 809ms/step - accuracy: 1.0000 - loss: 1.6889e-04 - val_accuracy: 1.00
Epoch 15/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 44s 847ms/step - accuracy: 1.0000 - loss: 1.4804e-04 - val_accuracy: 1.00
Epoch 16/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 44s 847ms/step - accuracy: 1.0000 - loss: 1.2906e-04 - val_accuracy: 1.00
Epoch 17/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 42s 816ms/step - accuracy: 1.0000 - loss: 1.1510e-04 - val_accuracy: 1.00
Epoch 18/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 81s 807ms/step - accuracy: 1.0000 - loss: 1.0115e-04 - val_accuracy: 1.00
Epoch 19/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 42s 807ms/step - accuracy: 1.0000 - loss: 8.9387e-05 - val_accuracy: 1.00
Epoch 20/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 83s 826ms/step - accuracy: 1.0000 - loss: 8.1347e-05 - val_accuracy: 1.00
52/52 ━━━━━━━━━━━━━━━━━━━━ 3s 62ms/step

Training Model B ...
Epoch 1/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 74s 1s/step - accuracy: 1.0000 - loss: 9.4274e-04 - val_accuracy: 1.0000
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 4s 84ms/step

Training Model C ...
Epoch 1/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 91s 2s/step - accuracy: 0.7512 - loss: 0.4949 - val_accuracy: 0.8332 - va
Epoch 2/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 91s 2s/step - accuracy: 0.9488 - loss: 0.1328 - val_accuracy: 1.0000 - va
Epoch 3/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 88s 2s/step - accuracy: 0.9995 - loss: 0.0084 - val_accuracy: 1.0000 - va
Epoch 4/20
52/52 ━━━━━━━━━━━━━━━━━━━━ 143s 2s/step - accuracy: 1.0000 - loss: 0.0019 - val_accuracy: 1.0000 - v
```

```
# ========================================
# Compare all models
# ========================================

results_df = pd.DataFrame(results)
results_df = results_df.sort_values(by="F1 Score", ascending=False).reset_index(drop=True)
results_df
```

	Model	Accuracy	Precision	Recall	F1 Score
0	A	1.000000	1.000000	1.000000	1.000000
1	B	0.969898	0.972944	0.969697	0.970027
2	C	0.833233	0.888351	0.836867	0.824954

Next steps: Generate code with `results_df` New interactive sheet

```
# Show results nicely in percentage
results_df_style = results_df.copy()
for col in ["Accuracy", "Precision", "Recall", "F1 Score"]:
    results_df_style[col] = results_df_style[col] * 100

print(results_df_style)
```

```
  Model    Accuracy   Precision      Recall    F1 Score
0     A  100.000000  100.000000  100.000000  100.000000
1     B   96.989765   97.294372   96.969697   97.002659
2     C   83.323299   88.835147   83.686690   82.495382
```

```
# Select best model based on F1 score
best_model_name = results_df.iloc[0]["Model"]
best_model = trained_models[best_model_name]

print("Best Model:", best_model_name)
best_model.summary()
```

```
Best Model: A
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
reshape_1 (Reshape)	(None, 128, 93, 1)	0
conv2d_2 (Conv2D)	(None, 128, 93, 8)	80
max_pooling2d_2 (MaxPooling2D)	(None, 64, 46, 8)	0
conv2d_3 (Conv2D)	(None, 64, 46, 16)	1,168
max_pooling2d_3 (MaxPooling2D)	(None, 32, 23, 16)	0
flatten_1 (Flatten)	(None, 11776)	0
dense_2 (Dense)	(None, 64)	753,728
dense_3 (Dense)	(None, 3)	195

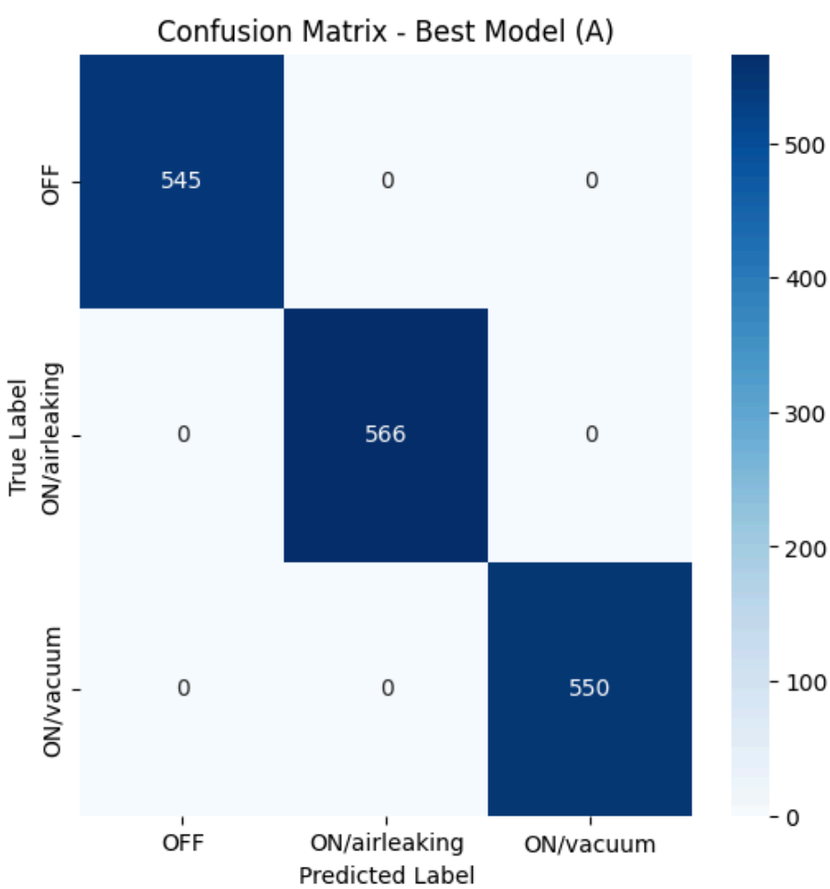
```
Total params: 2,265,515 (8.64 MB)
Trainable params: 755,171 (2.88 MB)
Non-trainable params: 0 (0.00 B)
```

```
# ========================================
# Evaluate best model
# ========================================
```

```
Y_pred_best = best_model.predict(X_test)
Y_pred_best_ind = np.argmax(Y_pred_best, axis=1)
Y_test_ind = np.argmax(Y_test, axis=1)

best_acc = accuracy_score(Y_test_ind, Y_pred_best_ind)
best_prec = precision_score(Y_test_ind, Y_pred_best_ind, average='macro')
best_rec = recall_score(Y_test_ind, Y_pred_best_ind, average='macro')
best_f1 = f1_score(Y_test_ind, Y_pred_best_ind, average='macro')

print("Best Model Performance")
print("Accuracy : {:.2f}%".format(best_acc * 100))
print("Precision: {:.2f}%".format(best_prec * 100))
print("Recall   : {:.2f}%".format(best_rec * 100))
print("F1 Score : {:.2f}%".format(best_f1 * 100))
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 3s 66ms/step
Best Model Performance
Accuracy : 100.00%
Precision: 100.00%
Recall   : 100.00%
F1 Score : 100.00%
```

```
# ==========================================
# Confusion Matrix for best model
# ==========================================

cm = confusion_matrix(Y_test_ind, Y_pred_best_ind)

plt.figure(figsize=(6,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=enc.classes_, yticklabels=enc.classes_)
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title(f"Confusion Matrix - Best Model ({best_model_name})")
plt.show()
```




```
# ==========================================
# Training curves for the best model
# ==========================================

history = histories[best_model_name]

plt.figure(figsize=(7,4))
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
```

```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title(f"Accuracy Curve - Model {best_model_name}")
plt.legend()
plt.show()

plt.figure(figsize=(7,4))
plt.plot(history.history['loss'], label='Train Loss')
```